



KARAR DESTEK SİSTEMLERİ

Numpy Kütüphanesi

Dr. Öğr. Üyesi. Neda ALIPOUR

nalipour@gelisim.edu.tr

İİSBF: Yönetim Bilişim Sistemleri

- NumPy (Numerical Python), bilimsel hesaplamalar için optimize edilmiş, güçlü bir Python kütüphanesidir. NumPy'nin temelinde N-dimensional array (ndarray) adı verilen, çok boyutlu veri yapılarını hızlı ve etkili bir şekilde işleyebilen bir yapı bulunur.
- **NOT:** NumPy, Python'da sayısal işlemler için kullanılır.
- **NOT:** Özellikle dizi (array) ve matris işlemlerini hızlı ve düzenli yapar.

- **AHP ve TOPSIS gibi çok kriterli karar verme yöntemlerinde sürekli olarak:**
 - **karar matrisi oluşturma,**
 - **normalize etme,**
 - **sütun toplamı alma,**
 - **ağırlıklarla çarpma,**
 - **maksimum ve minimum bulma**
- **gerekir.**
- **Bunların hepsi NumPy ile çok rahat yapılır.**

- `liste1 = [1, 2, 3]`
- `liste2 = [4, 5, 6]`
- `liste3 = liste1 + liste2`
- Normal liste oluşturursanız yukarıdaki toplamın sonucu `[1, 2, 3, 4, 5, 6]` ancak `liste1` ve `liste2` bir numpy listesine dönüştürülürse toplamın sonucu `[5, 7, 9]` olur.
- **Normal liste işleminde birleştirme numpy listesinde eleman bazlı toplama**

Başlat menüsüne yaz:

☞ **Jupyter Notebook**

Tıkla → tarayıcı açılır

Notebook oluşturma

Tarayıcı açıldığında:

Sağ üst:

☞ **New → Python 3**

Yeni sayfa açılır

- Numpy Kütüphanesini içe aktarmak için

```
[1]: import numpy as np
```

- Normal bir listeyi numpy arraya dönüştürmek

```
#Normal Liste
```

```
liste = [1, 2, 3]
```

```
liste
```

```
[1, 2, 3]
```

```
#numpy array
```

```
np_liste = np.array(liste)
```

```
np_liste
```

```
array([1, 2, 3])
```

- Numpy array içinde listeyi yazmak

```
a = np.array([1, 2, 3, 4, 5])
```

```
a
```

```
array([1, 2, 3, 4, 5])
```


- İç içe liste veya matris oluşturmak
- Parantez içinde köşeli parantez açılmalı
- 3*4 bir matris
- 3 satır ve 4 sütun

```
matris = np.array([  
    [1, 2, 3, 4],  
    [5, 6, 7, 8],  
    [9, 10, 11, 12]  
])  
matris
```

```
array([[ 1,  2,  3,  4],  
       [ 5,  6,  7,  8],  
       [ 9, 10, 11, 12]])
```

- `shape` fonksiyonu kullanılarak Matris boyut uzunluğu yazdırılabilir
- Yani kaç satır ve kaç sütundan oluşmuş

```
matris.shape
```

```
(3, 4)
```

- **ndim** matrisin kaç boyutlu olduğunu yazdırır
- **size** matriste kaç eleman olduğunu yazdırır

```
matris.ndim
```

2

```
matris.size
```

12

- Herhangi bir satıra erişmek için satırın index numarası yazılır.

- Birinci satır için:

```
matris[0]
```

```
array([[ 1,  2,  3,  4],  
       [ 5,  6,  7,  8],  
       [ 9, 10, 11, 12]])
```

```
array([1, 2, 3, 4])
```

- Birinci sütünü almak için:

```
matris=np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12]])  
matris[:,2]
```

```
array([ 3,  7, 11])
```

ELEMANA ERİŞMEK

- 2. satırda 3. elemana erişmek için

```
matris[1, 2]
```

7

```
array([[ 1,  2,  3,  4],  
       [ 5,  6,  7,  8],  
       [ 9, 10, 11, 12]])
```

Or

```
matris[1][2]
```

7

ELEMANA ERİŞMEK

- **Eleman aralıklarına erişmek için**
- **2. satırda 2. ile 4. eleman arasındakilere erişmek için**

```
array([[ 1,  2,  3,  4],  
       [ 5,  6,  7,  8],  
       [ 9, 10, 11, 12]])
```

```
matris[1][1:3]
```

```
array([6, 7])
```

```
matris[1, 1:3]
```

```
array([6, 7])
```

- **2. sutunda 2. ile 4. eleman arasındakilere erişmek için**

```
matris=np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12]])  
matris[1:3,1]
```

```
array([ 6, 10])
```

matris[:2,:2]

matris[1:,2:]

matris[:1,2:]

matris[1:,:1]

- Boyut belirleyerek 0'lardan oluşan array oluşturmak

```
tekBoyutSifirlar = np.zeros((3))  
matrisSifirlar = np.zeros((3, 5))  
  
print("Tek Boyutlu Sıfır Array:", tekBoyutSifirlar)  
  
print("İki Boyutlu Sıfır Array:\n", matrisSifirlar)
```

Tek Boyutlu Sıfır Array: [0. 0. 0.]

İki Boyutlu Sıfır Array:

[0. 0. 0. 0. 0.]

[0. 0. 0. 0. 0.]

[0. 0. 0. 0. 0.]

BİRLERLE ARRAY OLUŞTURMAK

- Boyut belirleyerek 1'lerden oluşan array oluşturmak

```
birler = np.ones((3, 4))
```

```
birler
```

```
array([[1., 1., 1., 1.],  
       [1., 1., 1., 1.],  
       [1., 1., 1., 1.]])
```

- Arange fonksiyonu ile belirli bir başlangıç ve bitiş arasında düzenli artışlarla bir array oluşturulur
- 0'dan 10'a kadar, 2 adımla

```
aralik = np.arange(0, 10, 2)  
aralik
```

```
array([0, 2, 4, 6, 8])
```

Örnek 1: 0'dan 9'a kadar bir array

```
aralik1 = np.arange(0, 10)
```

```
print("0'dan 9'a Array:", aralik1)
```

Örnek 2: Negatif aralıklı array

```
aralik2 = np.arange(10, 0, -2)
```

```
print("10'dan 0'a Negatif Adımlarla Array:", aralik2)
```

Örnek 3: Float değerlerle array

```
aralik3 = np.arange(0, 1, 0.2)
```

```
print("0'dan 1'e kesirli değerlerle Float Array:", aralik3)
```

```
0'dan 9'a Array: [0 1 2 3 4 5 6 7 8 9]
```

```
10'dan 0'a Negatif Adımlarla Array: [10  8  6  4  2]
```

```
0'dan 1'e kesirli değerlerle Float Array: [0.  0.2 0.4 0.6 0.8]
```

- **linspace** fonksiyonu ile başlangıç ve bitiş değerleri arasında belirtilen sayıda eşit aralıklarla değer üretir.

```
esitAralik = np.linspace(0, 1, 5)
```

```
esitAralik
```

```
array([0. , 0.25, 0.5 , 0.75, 1.  ])
```

```
esitAralik = np.linspace(0, 100, 5)
```

```
esitAralik
```

```
array([ 0., 25., 50., 75., 100.])
```

ARİTMETİK İŞLEMLER

- Toplama ve Çıkarma

```
a = np.array([1, 2, 3])  
b = np.array([8, 9, 10])
```

```
a + b
```

```
a - b
```

```
array([ 9, 11, 13])
```

```
array([-7, -7, -7])
```

- Çarpma ve Bölme

$a * b$

```
array([ 8, 18, 30])
```

a / b

```
array([0.125, 0.22222222, 0.3])
```

- Herhangi bir sayı ile toplama veya çarpma

$a + 10$

`array([11, 12, 13])`

$b * 5$

`array([40, 45, 50])`

- Matris Çarpımı

```
A = np.array([
    [1, 2],
    [3, 4]
])
B = np.array([
    [5, 6],
    [7, 8]
])
```

A@B

```
array([[19, 22],
       [43, 50]])
```


- **abs** fonksiyonu ile hesaplanır

```
a = np.array([1, -2, 3, -4])
```

```
np.abs(a)
```

```
array([1, 2, 3, 4])
```

- **power fonksiyonu ile hesaplanır**

```
a = np.array([1, -2, 3, -4])
```

```
np.power(a, 2)
```

```
array([ 1,  4,  9, 16], dtype=int32)
```

```
np.power(a, 3)
```

```
array([ 1, -8, 27, -64], dtype=int32)
```

- **** kullanımı**

```
a = np.array([1, -2, 3, -4])
```

```
a**2
```

```
array([ 1,  4,  9, 16])
```

```
a**3
```

```
array([ 1, -8, 27, -64], dtype=int32)
```

KAREKÖK ALMA

- **sqrt fonksiyonu ile hesaplanır**

```
matris = np.array([  
    [16, 25, 100],  
    [9, 36, 16],  
    [64, 49, 25]  
])
```

```
np.sqrt(matris)
```

```
array([[ 4.,  5., 10.],  
       [ 3.,  6.,  4.],  
       [ 8.,  7.,  5.]])
```

- **Matrisin Toplamı**

```
matris = np.array([  
    [5, 10, 11, 12],  
    [8, 3, 6, 9],  
    [12, 2, 9, 15]  
])
```

```
np.sum(matris)
```

102

- Matrisin Sütun Toplamı

```
matris = np.array([  
    [5, 10, 11, 12],  
    [8, 3, 6, 9],  
    [12, 2, 9, 15]  
)  
  
np.sum(matris, axis = 0)  
  
array([25, 15, 26, 36])
```

- Matrisin Satır Toplamı

```
matris = np.array([  
    [5, 10, 11, 12],  
    [8, 3, 6, 9],  
    [12, 2, 9, 15]  
)  
  
np.sum(matris, axis = 1)  
  
array([38, 26, 38])
```

TOPLAM İŞLEMİ

- Matrisin belirli bir sütununu toplamak için:
- `np.sum(matris ismi[:,sütün indexi])`
- Birinci indexte olan ikinci sütünün toplamını verir.

```
matris=np.array([  
[3,4,5],  
[6,7,9],  
[8,6,5]  
])  
np.sum(matris[:,1])
```

17

TOPLAM İŞLEMİ

- Matrisin belirli bir satırı toplamak için:
- `np.sum(matris ismi[satır indexi,:])`
- Birinci indexte olan ikinci satırın toplamını verir.

```
matris=np.array([  
    [3,4,5],  
    [6,7,9],  
    [8,6,5]  
])  
np.sum(matris[1,:])
```

22

- Tüm sütunları toplayıp hepsinin karekökünü bir liste olarak aynı anda almak
- Her sütunu kendi içinde topla, sonra hepsinin karekökünü al

```
matris=np.array([
    [1,2,3],
    [4,5,6],
    [7,8,9]
])
np.sqrt(np.sum(matris,axis=0))

array([3.46410162, 3.87298335, 4.24264069])
```

TOPLAM İŞLEMİ

- Tüm satırları toplayıp hepsinin karekökünü bir liste olarak aynı anda almak
- Her satırı kendi içinde topla, sonra hepsinin karekökünü al

```
matris=np.array([  
    [1,2,3],  
    [4,5,6],  
    [7,8,9]  
])  
np.sqrt(np.sum(matris,axis=1))
```

```
array([2.44948974, 3.87298335, 4.89897949])
```

- Matrisin Ortalaması

```
matris = np.array([  
    [5, 10, 11, 12],  
    [8, 3, 6, 9],  
    [12, 2, 9, 15]  
])
```

```
np.mean(matris)
```

8.5

- Matrisin Sütun Bazında Ortalaması

```
matris = np.array([  
    [5, 10, 11, 12],  
    [8, 3, 6, 9],  
    [12, 2, 9, 15]  
])
```

```
np.mean(matris, axis = 0)
```

```
array([ 8.33333333,  5.          ,  8.66666667, 12.          ])
```

- Matrisin Satır Bazında Ortalaması

```
matris = np.array([
    [5, 10, 11, 12],
    [8, 3, 6, 9],
    [12, 2, 9, 15]
])

np.mean(matris, axis = 1)

array([9.5, 6.5, 9.5])
```

ORTALAMA İŞLEMİ

- Matrisin belirli bir sütününün ortalamasını almak için:
- `np.mean(matris ismi[:,sütün indexi])`
- Birinci indexte olan ikinci sütünün ortalamasını verir.

```
matris=np.array([  
    [3,4,5],  
    [6,7,9],  
    [8,6,5]  
])  
np.mean(matris[:,1])
```

```
5.6666666666666667
```

ORTALAMA İŞLEMİ

- Matrisin belirli bir satırın ortalamasını almak için:
- `np.mean(matris ismi[satır indexi,:])`
- Birinci indexte olan ikinci satırın ortalamasını verir.

```
matris=np.array([  
    [3,4,5],  
    [6,7,9],  
    [8,6,5]  
])  
np.sum(matris[1,:])
```

22

- Matrisin Maksimumu

```
matris = np.array([  
    [5, 10, 11, 12],  
    [8, 3, 6, 9],  
    [12, 2, 9, 15]  
])
```

```
np.max(matris)
```

15

- Matrisin Sütun Bazında Maksimumu

```
matris = np.array([
    [5, 10, 11, 12],
    [8, 3, 6, 9],
    [12, 2, 9, 15]
])

np.max(matris, axis = 0)

array([12, 10, 11, 15])
```

- Matrisin Satır Bazında Maksimumu

```
matris = np.array([  
    [5, 10, 11, 12],  
    [8, 3, 6, 9],  
    [12, 2, 9, 15]  
)  
  
np.max(matris, axis = 1)  
  
array([12,  9, 15])
```

- Matrisin Minimumu

```
matris = np.array([  
    [5, 10, 11, 12],  
    [8, 3, 6, 9],  
    [12, 2, 9, 15]  
])
```

```
np.min(matris)
```

2

- Matrisin Sütun Bazında Minimumu

```
matris = np.array([  
    [5, 10, 11, 12],  
    [8, 3, 6, 9],  
    [12, 2, 9, 15]  
)  
  
np.min(matris, axis = 0)  
  
array([5, 2, 6, 9])
```

- Matrisin Satır Bazında Minimumu

```
matris = np.array([
    [5, 10, 11, 12],
    [8, 3, 6, 9],
    [12, 2, 9, 15]
])

np.min(matris, axis = 1)

array([5, 3, 2])
```

MİN ve MAX İŞLEMİ

- Matrisin belirli bir satırın Min veya Maxini bulmak için:
- `np.min(matris ismi[satır indexi,:])`
- Birinci indexte olan ikinci satırın minimumunu verir.

`np.max(matris ismi[satır indexi,:])`

```
matris=np.array([[5,10,2],[7,9,18],[15,10,30]])  
np.min(matris[1,:])
```

7

```
matris=np.array([[5,10,2],[7,9,18],[15,10,30]])  
np.max(matris[0,:])
```

10

MİN ve MAX İŞLEMİ

- Matrisin belirli bir sütünün Min veya Maxini bulmak için:
- `np.min(matris ismi[:, sutun indexi])`
- İkinci indexte olan üçüncü sütünün maximumunu verir.

```
matris=np.array([[5,10,2],[7,9,18],[15,10,30]])  
np.max(matris[:,2])
```

30